

Article

A Secure and Efficient ECC-Based User-to-User Authentication and Key Agreement Protocol for Smart Grid

Yujin Nam, Hyewon Park  and Yohan Park * 

School of Computer Engineering, Keimyung University, Daegu 42601, Republic of Korea; readaworld369@stu.kmu.ac.kr (Y.N.); wldnjsfuf@stu.kmu.ac.kr (H.P.)

* Correspondence: yhpark@kmu.ac.kr; Tel.: +82-53-580-5229

Abstract

Smart grid environments increasingly require secure user-to-user (U2U) communication for decentralized applications such as peer-to-peer energy trading and electric vehicle energy exchange. However, many existing authentication and key agreement schemes rely on centralized entities during authentication or provide insufficient protection against impersonation, insider, and ephemeral secret leakage (ESL) attacks. To address these limitations, this paper proposes an Elliptic Curve Cryptography (ECC)-based secure U2U authentication and key agreement protocol for smart grid environments. The proposed protocol employs ECDH-based session key establishment and Schnorr verification to enable mutual authentication without directly exposing long-term credentials. In addition, the protocol provides user anonymity and unlinkability by using session-dependent identity protection mechanisms. Security analysis demonstrates that the proposed scheme can resist various attacks, including impersonation, replay, denial-of-service, privileged insider, stolen device, and ESL attacks. Furthermore, simulation-based comparative analysis shows that the proposed protocol achieves stronger security features than several existing schemes, although it incurs moderately higher computational and communication costs due to additional elliptic curve operations and verification parameters.

Keywords: smart grid; user-to-user communication; authentication and key agreement; elliptic curve cryptography; Schnorr verification

1. Introduction

Smart grids (SGs) integrate information and communication technology (ICT) into power infrastructures to support efficient and intelligent energy management [1,2]. Compared with traditional power grids, which mainly rely on centralized control and one-way communication, SGs support bidirectional communication, distributed energy generation, and real-time monitoring. These capabilities have enabled decentralized applications such as peer-to-peer energy trading, vehicle-to-grid systems, and distributed energy sharing, especially with the growing use of electric vehicles (EVs), renewable energy resources, and smart devices [3,4]. In these settings, direct communication among users and devices is essential for autonomous energy exchange and coordination. As SG entities communicate more frequently, secure authentication and key agreement (AKA) mechanisms become necessary. Applications such as user-to-user (U2U) energy trading and vehicle-to-vehicle energy exchange require entities to authenticate each other over public channels [5–9], often without relying on centralized infrastructure during communication. This creates



Academic Editors: Shuo Zhang,
Guibin Zou and Xiuyan Wei

Received: 15 May 2026

Revised: 9 June 2026

Accepted: 12 June 2026

Published: 15 June 2026

Copyright: © 2026 by the authors.
Licensee MDPI, Basel, Switzerland.
This article is an open access article
distributed under the terms and
conditions of the [Creative Commons
Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

several security risks, including impersonation, replay, man-in-the-middle (MITM), denial-of-service (DoS), and privileged insider attacks [2,10]. Privacy is also a major concern, since electricity consumption data can reveal users' behaviors and lifestyles [11]. Thus, decentralized SG environments require lightweight AKA protocols that can provide secure and privacy-preserving communication with low overhead.

To address these challenges, several AKA schemes have been proposed for SG systems. Chim et al. [12] introduced a privacy-preserving authentication scheme, while Nicanfar and Leung [13] proposed an ECC-based AKA protocol for SG environments. Subsequent studies, including those by Odelu et al. [14], Wazid et al. [15], Gope and Sikdar [16], Hu et al. [17], and Wu et al. [18], further improved the security and efficiency of ECC-based authentication for resource-constrained devices. However, most existing schemes still follow a centralized model in which a trusted authority or service provider is involved in authentication. As a result, they are not fully suitable for decentralized SG applications, where entities must establish trust and session keys directly over public channels without relying on online assistance from a trusted third party. Recently, Khan et al. [19] proposed an ECC-based AKA protocol for smart grid communication environments (SGAK). Although SGAK aims to support mutual authentication and secure session-key establishment, our cryptanalysis identifies several security weaknesses. Specifically, SGAK is vulnerable to impersonation, ephemeral secret leakage (ESL), and DoS attacks. In addition, it does not sufficiently ensure anonymity, untraceability, data confidentiality, or message authentication.

To overcome these limitations, this paper proposes a lightweight ECC-based AKA protocol for U2U smart grid communications. In the proposed scheme, users can authenticate each other directly and establish a secure session key without requiring a trusted authority to participate online during authentication. The protocol is designed to resist impersonation, replay, privileged insider, and ESL attacks. It also preserves user anonymity and untraceability, making it suitable for decentralized SG applications. The main contributions of this paper are summarized as follows:

- We analyze Khan et al.'s SGAK protocol and identify several security weaknesses, including susceptibility to impersonation, ESL, and DoS attacks.
- We propose a lightweight ECC-based AKA protocol for decentralized communication in smart grid environments, where users can authenticate each other without online support from a trusted authority.
- The proposed scheme provides mutual authentication and secure session-key establishment, and is designed to resist replay, impersonation, privileged insider, and ESL attacks.
- We validate the security and efficiency of the proposed protocol through formal and informal security analysis, as well as simulation-based performance comparison with existing schemes.

2. Related Works

Since Fouda et al. [20] proposed a computational Diffie–Hellman-based AKA protocol for smart grid (SG) communication in 2011, numerous authentication and key agreement schemes have been developed for SG environments. Although Fouda et al.'s scheme established an early foundation for SG security, its reliance on exponential operations resulted in relatively high computation and communication costs, limiting its applicability to resource-constrained environments. To address privacy concerns in SG systems, Chim et al. [12] introduced a privacy-preserving authentication scheme; however, the protocol does not fully provide key agreement, mutual authentication, or perfect forward secrecy (PFS), and may be vulnerable to impersonation and insider attacks. Subsequently, Nicanfar and Leung [13] proposed an ECC-based key agreement protocol for access control between

home area network devices and external controllers, supporting forward and backward confidentiality. Nevertheless, the scheme does not fully satisfy the increasing security demands of modern SG environments.

Building on these studies, Tsai and Lo [21] proposed an anonymous key distribution authentication scheme for SG systems and highlighted vulnerabilities in prior approaches, including replay and impersonation attacks as well as the lack of PFS. However, subsequent analysis revealed that their scheme may not guarantee efficient credential privacy for smart meters (SMs) or secure session key establishment. To overcome these limitations, Odelu et al. [14] presented a provably secure AKA scheme with reduced computational overhead. Despite this improvement, later studies suggested that the protocol may still be vulnerable to impersonation and man-in-the-middle (MITM) attacks and may not fully consider the practical constraints of real SM devices. In subsequent works, Wazid et al. [15] proposed a three-factor authentication protocol for renewable energy-based SG communication, while Abbasinezhad-Mood and Nikooghadam [22] introduced a lightweight ECC-based authentication scheme suitable for practical SG environments. Gope and Sikdar [16] further incorporated physically unclonable functions (PUFs) [23,24] to enhance privacy preservation and resistance against DoS and MITM attacks. Nevertheless, these schemes still suffer from limitations such as incomplete anonymity support and practical security concerns associated with fuzzy extractors.

More recently, Srinivas et al. [25] proposed an anonymous signature-based authenticated key exchange scheme for IoT-enabled SG systems using Schnorr signatures, aiming to provide anonymity and efficient key exchange; however, the scheme may not effectively resist ESL attacks. Hu et al. [17] later introduced an ECC-based AKA protocol between SMs and server virtual nodes with open-channel registration and provable security claims. Wu et al. [18] subsequently demonstrated that Hu et al.'s protocol is vulnerable to key compromise impersonation (KCI) attacks and fails to provide untraceability, and then proposed a security-enhanced AKA scheme based on the elliptic curve Menezes–Qu–Vanstone (ECMQV) mechanism. Similarly, Nkenyereye et al. [26] proposed LAP-SG, a lightweight ECC-based authentication scheme designed for resource-constrained SM devices in large-scale SG networks, reducing computation, communication, and storage overhead compared with existing approaches.

Despite these advancements, existing protocols still exhibit limitations in terms of security robustness and practical deployment efficiency. In particular, Khan et al. [19] recently proposed SGAK; however, our cryptanalysis reveals that SGAK suffers from several critical security weaknesses. Specifically, the scheme is vulnerable to impersonation, ESL, and DoS attacks while failing to fully guarantee data confidentiality, user anonymity, untraceability, and message authentication. These limitations motivate the design of the proposed protocol in this work.

3. Preliminaries

In this section, we briefly review the fundamental cryptographic notions required for the understanding of the proposed protocol, including elliptic curve cryptography, the elliptic curve Diffie–Hellman problem, and a Schnorr-based signature mechanism.

3.1. Elliptic Curve Cryptography

ECC is a public-key cryptographic technique that relies on the algebraic properties of elliptic curves defined over finite fields [27]. In comparison with conventional public-key systems such as RSA, ECC can provide an equivalent level of security while using significantly smaller key sizes. This characteristic makes ECC particularly appropriate for

devices with limited computational resources, such as smart meters deployed in smart grid environments.

An elliptic curve E defined over a prime finite field $\mathbb{Z}_p$ can be represented by the following Weierstrass equation:

$$E : y^2 = x^3 + ax + b \pmod{p} \quad (1)$$

where p denotes a large prime number, and $a, b \in \mathbb{Z}_p$. The curve must satisfy the non-singularity condition $4a^3 + 27b^2 \pmod{p} \neq 0$ to ensure that it forms a valid elliptic curve. The collection of all points satisfying the above equation, together with the point at infinity $\mathcal{O}$, constitutes an abelian group $\mathbb{G}$ under elliptic curve point addition, where $\mathcal{O}$ acts as the identity element. Let G be a publicly known generator point of prime order q , such that $q \cdot G = \mathcal{O}$. For an integer $k \in \mathbb{Z}_q^*$, scalar multiplication is defined as repeated addition of the generator point, i.e., $k \cdot G = G + G + \dots + G$ performed k times [27].

In the proposed protocol, elliptic curve scalar multiplication serves as the core operation for generating temporary key pairs and establishing shared secret values during the authentication and key agreement procedure.

3.2. Elliptic Curve Diffie–Hellman Problem

The security of cryptographic constructions based on elliptic curves is primarily derived from the computational difficulty of two well-known problems: the Elliptic Curve Discrete Logarithm Problem (ECDLP) and the Elliptic Curve Diffie–Hellman Problem (ECDHP) [27]. The ECDLP states that, given a generator point G of prime order q and another point $Q = k \cdot G \in \mathbb{G}$, it is computationally infeasible to recover the scalar value $k \in \mathbb{Z}_q^*$ within polynomial time. Based on this assumption, the ECDHP can be described as follows. Given two publicly known points $a \cdot G$ and $b \cdot G$ corresponding to unknown integers $a, b \in \mathbb{Z}_q^*$, it is computationally infeasible to compute the point $a \cdot b \cdot G$ without knowledge of either a or b . Formally, for any probabilistic polynomial-time adversary $\mathcal{A}$, the advantage of solving the ECDHP is defined as:

$$\text{Adv}_{\mathcal{A}}^{\text{ECDHP}} = \Pr[\mathcal{A}(G, a \cdot G, b \cdot G) = a \cdot b \cdot G] \leq \varepsilon \quad (2)$$

where ε represents a negligible positive value.

In the context of this work, the hardness of the ECDLP ensures that temporary secret values cannot be recovered from publicly exchanged parameters, whereas the hardness of the ECDHP guarantees that the session key established through the ECDH-based key agreement process remains secure against external adversaries.

3.3. Schnorr-Based Signature

The Schnorr signature [28] is a lightweight digital signature technique whose security relies on the intractability of the discrete logarithm problem. It follows a three-step interaction pattern consisting of a commitment, a challenge, and a response, collectively enabling a zero-knowledge proof of knowledge of a secret value.

Let $\mathbb{G}$ be a cyclic group of prime order q with generator point G . Assume that a prover possesses a long-term private key $\sigma \in \mathbb{Z}_q^*$ and a corresponding public key $X = \sigma \cdot G$. To demonstrate knowledge of the secret value σ without revealing it, the prover executes the following procedure. First, the prover selects a random nonce $u \in \mathbb{Z}_q^*$ and computes a commitment $M = u \cdot G$. Next, a challenge value is generated as $\text{Auth} = h(M \parallel \cdot)$, where

$h(\cdot)$ denotes a cryptographic hash function and the input may include session-related parameters. Finally, the prover computes a response value:

$$z = u + Auth \cdot \sigma \pmod{q} \quad (3)$$

The verifier validates the correctness of the signature by checking whether

$$z \cdot G \stackrel{?}{=} M + Auth \cdot X \quad (4)$$

This equality holds because

$$z \cdot G = (u + Auth \cdot \sigma) \cdot G = u \cdot G + Auth \cdot \sigma \cdot G = M + Auth \cdot X \quad (5)$$

The zero-knowledge property of this mechanism ensures that the response value z does not reveal any information about the private key σ , since the nonce u is independently and randomly generated for each execution.

In the proposed protocol, a Schnorr-based signature is incorporated into the authentication phase to enable each smart meter to prove possession of its long-term credential σ_i without exposing it over public communication channels, thereby providing protection against credential compromise and impersonation attacks.

3.4. System Model

The proposed U2U smart grid communication environment consists of two types of entities: users and a trusted authority (TA). Two users communicate directly with each other over a public channel after completing the registration process with the trusted authority. The trusted authority is responsible for managing system parameters and assisting in the registration of users. Finally, both users establish a shared session key for secure communication. The details of each entity are described below and illustrated in Figure 1.

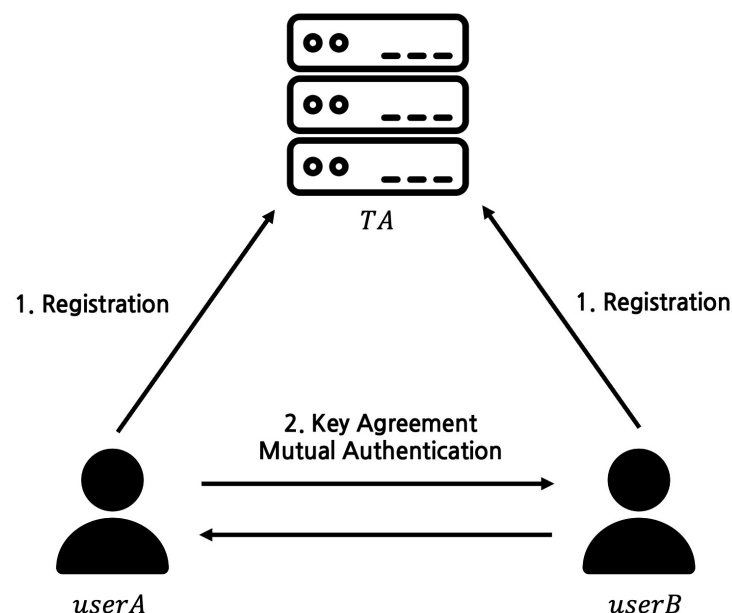


Figure 1. System model.

- **User (U_i):** A user U_i represents a smart grid device participating in U2U communication with other registered users in the smart grid environment. The user is equipped with communication and computation capabilities to perform authentication and secure communication. Before initiating communication, the user must complete a

one-time registration with the trusted authority through a secure channel. After registration, the user stores necessary parameters in its device memory and performs mutual authentication and session key establishment with other registered users over a public channel. We assume that the user's device may be compromised or controlled by an adversary.

- TA: The TA is responsible for system initialization, parameter generation, and user registration management. Specifically, the TA generates system parameters, verifies the legitimacy of users during the registration phase, and securely distributes necessary credentials. We assume that the trusted authority is a trusted entity and operates in a secure environment.

3.5. Adversary Model

We employ an adversary model based on the Canetti–Krawczyk (CK) model [29] to define the capabilities of the adversary.

The details are as follows:

- The adversary can eavesdrop on, intercept, modify, inject, delay, or replay messages transmitted over public channels.
- The adversary can extract stored parameters from a user's device if the device is physically captured or compromised.
- The adversary can obtain session-specific ephemeral secrets or intermediate parameters of compromised sessions.
- The adversary can launch various attacks, including impersonation, replay, and man-in-the-middle attacks.
- The adversary may also operate as an insider within the system.

3.6. Notations

This subsection summarizes the main symbols and notations used throughout the proposed protocol, as listed in Table 1.

Table 1. Notations.

Symbol	Description
U_i	User
TA	Trusted authority
ID_i	Identity of U_i
PID_i	Pseudonym identity
x_i	Secret value
X_i	Public key
s	Master secret key of TA
σ_i	Long-term secret credential
W_i	Verification parameter
r_A, r_B	Random numbers
T_1, T_2	Timestamp
K_{AB}, K_{BA}	Ephemeral shared secret
SK	Session key
$h(\cdot)$	Hash function
$\oplus$	Exclusive-or operation
$\parallel$	Concatenation operation
g	Generator point
q	Order of g

4. Revisiting Khan et al.'s Scheme

4.1. Initialization Phase

In this phase, the TA initializes the system parameters and publishes the public parameters $\{E, g, q, h(\cdot)\}$ for use in the subsequent registration and authentication phases.

4.2. Registration Phase

Before taking part in the authentication process, each user must register with the TA through a secure channel. The registration steps are summarized as follows.

- Step 1: U_i selects ID_i and TR_1 , and transmits (ID_i, TR_1) to the TA through a secure channel.
- Step 2: Upon receiving (ID_i, TR_1) , the TA checks the freshness condition and generates $x_i \in Z_q^*$. It then computes $X_i = x_i \cdot g$, $d_i = h(ID_i \| X_i)$, and $Y_i = x_i + M \cdot d_i$, and sends (Y_i, X_i) to U_i through a secure channel.
- Step 3: Upon receiving (Y_i, X_i) , U_i verifies whether $Y_i \cdot g \stackrel{?}{=} X_i + h(ID_i \| X_i) \cdot PK$. If valid, U_i sets $PK_i = Y_i \cdot g$.

4.3. Login and Authentication Phase

In this phase, two users perform mutual authentication and establish a shared session key for secure communication. The detailed message flow is illustrated in Figure 2.

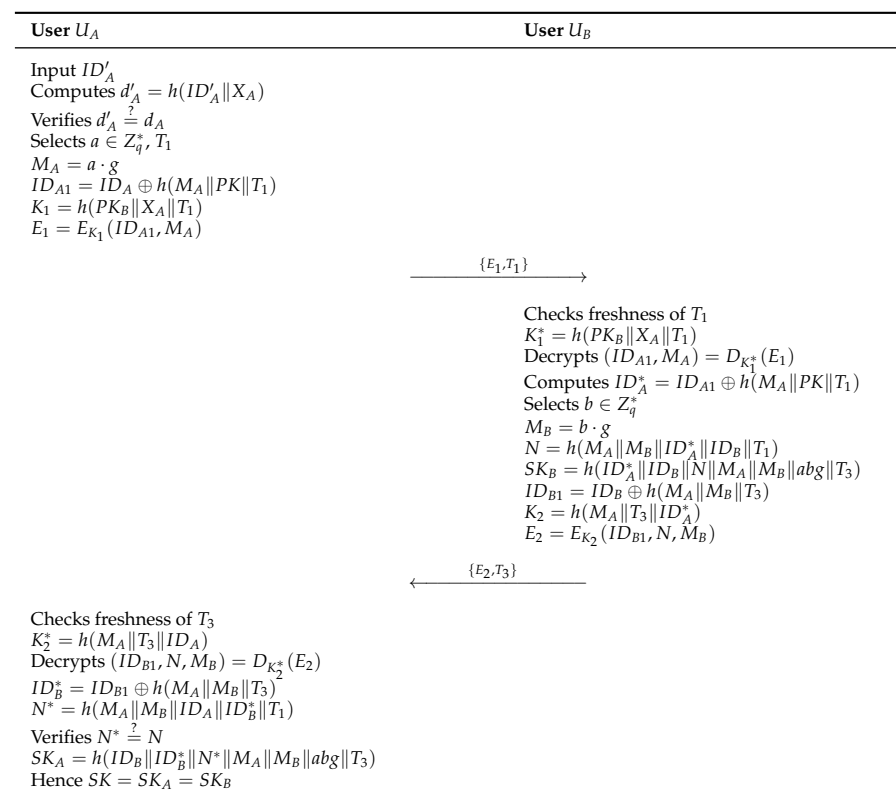


Figure 2. Login and authentication phase of Khan et al.'s scheme.

- Step 1: U_A inputs ID'_A , computes $d'_A = h(ID'_A \| X_A)$ and verifies whether $d'_A = d_A$. If valid, U_A selects a random number $a \in Z_q^*$ and timestamp T_1 , computes $M_A = a \cdot g$, $ID_{A1} = ID_A \oplus h(M_A \| PK \| T_1)$, and $E_1 = E_{K_1}(ID_{A1}, M_A)$, where $K_1 = h(PK_B \| X_A \| T_1)$. Then U_A sends (E_1, T_1) to U_B over a public channel.
- Step 2: Upon receiving (E_1, T_1) , U_B checks the freshness of the timestamp. U_B decrypts (ID_{A1}, M_A) using $K_1^* = h(PK_B \| X_A \| T_1)$ and computes $ID_A^* = ID_{A1} \oplus h(M_A \|$

$PK \parallel T_1$). Then U_B selects a random number $b \in Z_q^*$ and computes $M_B = b \cdot g$ and $N = h(M_A \parallel M_B \parallel ID_A^* \parallel ID_B \parallel T_1)$. Next, U_B derives the session key $SK_B = h(ID_A^* \parallel ID_B \parallel N \parallel M_A \parallel M_B \parallel abg \parallel T_3)$, computes $ID_{B1} = ID_B \oplus h(M_A \parallel M_B \parallel T_3)$, and $E_2 = E_{K_2}(ID_{B1}, N, M_B)$, where $K_2 = h(M_A \parallel T_3 \parallel ID_A^*)$. Finally, U_B sends (E_2, T_3) to U_A over a public channel.

Step 3: Upon receiving (E_2, T_3) , U_A checks the freshness of the timestamp and computes $K_2^* = h(M_A \parallel T_3 \parallel ID_A)$ to decrypt (ID_{B1}, N, M_B) . Then U_A computes $ID_B^* = ID_{B1} \oplus h(M_A \parallel M_B \parallel T_3)$ and $N^* = h(M_A \parallel M_B \parallel ID_A \parallel ID_B^* \parallel T_1)$, and verifies $N^* = N$. If valid, U_A derives the session key $SK_A = h(ID_B \parallel ID_B^* \parallel N^* \parallel M_A \parallel M_B \parallel abg \parallel T_3)$, and both users obtain the same session key, i.e., $SK = SK_A = SK_B$.

4.4. Key Update Phase

To maintain long-term security, the protocol provides a key update mechanism that allows users to refresh their cryptographic keys when necessary. The key update procedure follows a process similar to the authentication phase and establishes a new session key.

Step 1: U_A selects a new private value $a_{new} \in Z_q^*$ and timestamp T_1^{new} , computes $M_A^{new} = a_{new} \cdot g$, and generates the encrypted message E_1^{new} in the same manner as in the authentication phase, where $K_1^{new} = h(PK_B \parallel X_A \parallel T_1^{new})$. Then U_A sends (E_1^{new}, T_1^{new}) to U_B over a public channel.

Step 2: Upon receiving (E_1^{new}, T_1^{new}) , U_B checks the freshness of the timestamp and decrypts $(ID_{A1}^{new}, M_A^{new})$ using $K_1^{new} = h(PK_B^{new} \parallel X_A^{new} \parallel T_1^{new})$. Then U_B computes $ID_A^{*new} = ID_{A1}^{new} \oplus h(M_A^{new} \parallel PK_B^{new} \parallel T_1^{new})$, selects $b_{new} \in Z_q^*$, computes $M_B^{new} = b_{new} \cdot g$ and $N^{new} = h(M_A^{new} \parallel M_B^{new} \parallel ID_A^{*new} \parallel ID_B^{new} \parallel T_1^{new})$, derives $SK_B^{new} = h(ID_A^{*new} \parallel ID_B^{new} \parallel N^{new} \parallel M_A^{new} \parallel M_B^{new} \parallel a_{new}b_{new}g \parallel T_3^{new})$, computes $ID_{B1}^{new} = ID_B \oplus h(M_A^{new} \parallel M_B^{new} \parallel T_3^{new})$, and generates $E_2^{new} = E_{K_2^{new}}(ID_{B1}^{new}, N^{new}, M_B^{new})$, where $K_2^{new} = h(T_3^{new} \parallel ID_A^{*new})$. Finally, U_B sends (E_2^{new}, T_3^{new}) to U_A over a public channel.

Step 3: Upon receiving (E_2^{new}, T_3^{new}) , U_A checks the freshness of the timestamp and decrypts the received message. The subsequent verification and session key derivation are performed in the same manner as in Step 3 of the authentication phase. If the verification is successful, both users obtain the same updated session key, i.e., $SK^{new} = SK_A^{new} = SK_B^{new}$.

5. Cryptanalysis of Khan et al.'s Scheme

In this section, we conduct a thorough cryptanalysis of Khan et al.'s scheme and demonstrate that it is susceptible to several security attacks. Specifically, we show that the scheme fails to prevent impersonation attack, ESL attack, and DoS attack, and cannot ensure data confidentiality, user anonymity, untraceability, and message authentication. The detailed steps of each attack are described in the following subsections.

5.1. Impersonation Attack

A malicious adversary $\mathcal{A}$ eavesdrops on the public channel and impersonates a legitimate user U_A to establish a session key with U_B . The detailed steps are outlined as follows.

Step 1: $\mathcal{A}$ intercepts the message $\{E_1, T_1\}$ transmitted from U_A to U_B over the public channel. Since PK_B , X_A , and T_1 are all public values, $\mathcal{A}$ can directly compute $K_1 = h(PK_B \parallel X_A \parallel T_1)$.

Step 2: $\mathcal{A}$ decrypts E_1 using K_1 to obtain ID_{A1} and M_A . Then, $\mathcal{A}$ recovers the real identity of U_A by computing $ID_A = ID_{A1} \oplus h(M_A \parallel PK \parallel T_1)$.

- Step 3: $\mathcal{A}$ selects a fresh random value a' and computes $M'_A = a' \cdot g$. Using the recovered ID_A , $\mathcal{A}$ generates $ID'_{A1} = ID_A \oplus h(M'_A \| PK \| T'_1)$ and transmits a forged message $\{E'_1, T'_1\}$ to U_B , encrypted with $K'_1 = h(PK_B \| X_A \| T'_1)$, impersonating U_A .
- Step 4: U_B accepts the forged message as a legitimate request from U_A . U_B generates $M_B = b \cdot g$, computes the session key SK_B , and transmits an encrypted response message E_2 to $\mathcal{A}$.
- Step 5: $\mathcal{A}$ computes $K_2 = h(M'_A \| T_3 \| ID_A)$ to decrypt E_2 and obtains ID_B , N , and M_B . $\mathcal{A}$ then fully recovers SK_B by computing $b \cdot a' \cdot g = a' \cdot M_B$. Consequently, U_B shares a session key with $\mathcal{A}$ while mistakenly believing it is communicating with the legitimate U_A , and the real U_A remains completely unaware of this fact.

5.2. Ephemeral Secret Leakage Attack

A malicious adversary $\mathcal{A}$ assumes that the ephemeral secret b has been leaked and combines it with information obtained by eavesdropping on the public channel to fully recover the session key SK_B . The detailed steps are outlined as follows.

- Step 1: $\mathcal{A}$ computes $K_1 = h(PK_B \| X_A \| T_1)$ using the public values PK_B , X_A , and T_1 , and decrypts the intercepted E_1 to obtain ID_{A1} and M_A . Then, $\mathcal{A}$ recovers the real identity of U_A by computing $ID_A = ID_{A1} \oplus h(M_A \| PK \| T_1)$.
- Step 2: $\mathcal{A}$ computes $K_2 = h(M_A \| T_3 \| ID_A^*)$ using M_A , ID_A , and T_3 , and decrypts the intercepted E_2 to obtain ID_B , N , and M_B .
- Step 3: Once b is leaked, $\mathcal{A}$ can fully recover the session key by computing $b \cdot a \cdot g = b \cdot M_A$, and thus derives $SK_B = h(ID_A^* \| ID_B \| N \| M_A \| M_B \| b \cdot a \cdot g \| T_3)$. This directly contradicts Khan et al.'s security claim that both the long-term and ephemeral secrets must be known to compute the session key.

5.3. Data Confidentiality Violation

Khan et al.'s scheme claims that data confidentiality is guaranteed by encrypting E_1 with $K_1 = h(PK_B \| X_A \| T_1)$. However, since PK_B , X_A , and T_1 are all public values exposed over the public channel, any adversary $\mathcal{A}$ can directly compute K_1 and decrypt E_1 to obtain ID_{A1} and M_A . Similarly, $K_2 = h(M_A \| T_3 \| ID_A)$ can also be computed using the previously obtained M_A and the public values T_3 and ID_A , allowing $\mathcal{A}$ to decrypt E_2 as well. Consequently, Khan et al.'s claim that "an adversary cannot decrypt the messages" does not hold.

5.4. User Anonymity and Untraceability Violation

Khan et al.'s scheme claims that user anonymity and untraceability are guaranteed since U_A transmits its anonymous identity ID_{A1} in encrypted form. However, since all values composing $K_1 = h(PK_B \| X_A \| T_1)$, namely PK_B , X_A , and T_1 , are exposed over the public channel, $\mathcal{A}$ can directly compute K_1 , decrypt E_1 , and recover the real identity ID_A . Furthermore, by repeatedly recovering ID_A from different sessions using the same method, $\mathcal{A}$ can determine that those sessions originated from the same user. Therefore, both user anonymity and untraceability do not hold in Khan et al.'s scheme.

5.5. Message Unauthentication

Khan et al.'s scheme claims that message authentication is guaranteed through timestamp and hash value verification. However, when U_B receives E_1 , it only checks the validity of the timestamp and has no means to verify whether the message actually originated from U_A . Therefore, even if $\mathcal{A}$ transmits a forged message, U_B accepts it as a legitimate message from U_A . Consequently, Khan et al.'s scheme fails to guarantee message origin authentication.

5.6. Denial-of-Service Attack

Khan et al.'s scheme claims that DoS attacks are prevented through the verification $d_A^* \stackrel{?}{=} d_A$. However, since d_A is not delivered to U_A during the registration phase, this verification cannot be performed in practice. Furthermore, U_B performs M_B generation, SK_B computation, and E_2 construction upon receiving E_1 without any prior validation. Therefore, if $\mathcal{A}$ transmits a large number of forged messages $\{E_1', T_1'\}$, U_B 's computational resources are exhausted. Consequently, Khan et al.'s claim of DoS resistance does not hold.

6. Proposed Scheme

In this section, we propose a mutual authentication and key agreement protocol for U2U smart grid communication based on elliptic curve cryptography. The proposed scheme consists of four phases: system initialization, registration, authentication and key agreement, and re-authentication. The flowcharts of each phase are illustrated in the corresponding figures.

6.1. Initialization Phase

Before the registration phase, the trusted authority (TA) establishes the system parameters as follows. The TA selects an elliptic curve E over a finite field $\mathbb{F}_p$ with a base point g of prime order q , chooses a master secret key s , and computes the corresponding public key $P_s = s \cdot g$. The TA also selects a secure one-way hash function $h(\cdot)$ and publishes the system parameters $\{E, g, q, h(\cdot), P_s\}$ over a public channel, while keeping s secret.

6.2. Registration Phase

Before participating in the authentication process, each user U_i must complete a one-time registration with the TA through a secure channel. The detailed process is outlined below and illustrated in Figure 3.

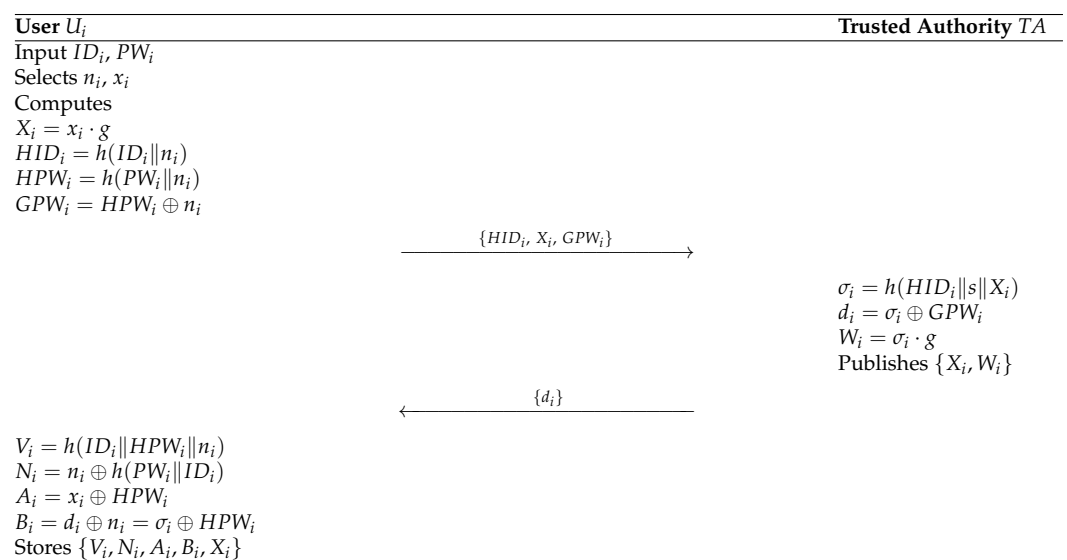


Figure 3. Registration Phase.

- Step 1: U_i inputs ID_i and PW_i , and selects random values n_i and x_i . U_i then computes $X_i = x_i \cdot g$, $HID_i = h(ID_i || n_i)$, $HPW_i = h(PW_i || n_i)$, and $GPW_i = HPW_i \oplus n_i$, and sends $\{HID_i, X_i, GPW_i\}$ to the TA via a secure channel.
- Step 2: After receiving the registration request, the TA computes $\sigma_i = h(HID_i || s || X_i)$, $d_i = \sigma_i \oplus GPW_i$, and $W_i = \sigma_i \cdot g$. The TA publishes $\{X_i, W_i\}$ on a public channel and sends $\{d_i\}$ back to U_i via a secure channel.

Step 3: After receiving $\{d_i\}$ from the TA, U_i computes $V_i = h(ID_i \| HPW_i \| n_i)$, $N_i = n_i \oplus h(PW_i \| ID_i)$, $A_i = x_i \oplus HPW_i$, and $B_i = d_i \oplus n_i = \sigma_i \oplus HPW_i$, and stores $\{V_i, N_i, A_i, B_i, X_i\}$ in its device memory.

6.3. Authentication and Key Agreement Phase

Once registration is complete, U_A and U_B can mutually authenticate each other and establish a shared session key. The detailed process is as follows and is illustrated in Figure 4.



Figure 4. Authentication and Key Agreement Phase.

Step 1: U_A inputs ID_A and PW_A , computes $n_A = N_A \oplus h(PW_A \| ID_A)$, $HPW_A = h(PW_A \| n_A)$, $V'_A = h(ID_A \| HPW_A \| n_A)$, and verifies the user via $V_A \stackrel{?}{=} V'_A$. U_A then selects random values r_A , u_A , and timestamp T_1 , and computes $M_A = r_A \cdot g$, $T_A = u_A \cdot g$, $x_A = A_A \oplus HPW_A$, $\sigma_A = B_A \oplus HPW_A$, $K_{AB} = h(r_A \cdot X_B \| T_1)$, $Auth_A = h(ID_A \| ID_B \| M_A \| K_{AB} \| r_A \| T_1)$, $z_A = u_A + Auth_A \cdot \sigma_A \pmod{q}$, $r_{A_{mask}} = r_A \oplus h(K_{AB} \| T_1)$, and $PID_A = ID_A \oplus h(K_{AB} \| r_A \| T_1)$. Finally,

U_A sends the message $M_1 = \{PID_A, M_A, z_A, T_A, r_{A_{mask}}, T_1\}$ to U_B over a public channel.

- Step 2: After receiving M_1 from U_A , U_B checks the timestamp $|T_{now} - T_1| \leq \Delta T$. If the timestamp is valid, U_B inputs ID_B and PW_B , computes $n_B = N_B \oplus h(PW_B \| ID_B)$, $HPW_B = h(PW_B \| n_B)$, $V'_B = h(ID_B \| HPW_B \| n_B)$, and authenticates the user via $V_B \stackrel{?}{=} V'_B$. U_B then computes $x_B = A_B \oplus HPW_B$, $\sigma_B = B_B \oplus HPW_B$, $K_{AB} = h(x_B \cdot M_A \| T_1)$, $r_A = r_{A_{mask}} \oplus h(K_{AB} \| T_1)$, $ID_A = PID_A \oplus h(K_{AB} \| r_A \| T_1)$, and $Auth_A^* = h(ID_A \| ID_B \| M_A \| K_{AB} \| r_A \| T_1)$, and uses $z_A \cdot g \stackrel{?}{=} T_A + Auth_A^* \cdot W_A$ to verify that message M_1 came from user U_A . U_B then selects random values r_B , u_B , and timestamp T_2 , and computes $M_B = r_B \cdot g$, $T_B = u_B \cdot g$, $K_{BA} = h(r_B \cdot X_A \| T_2)$, $PID_B = ID_B \oplus h(K_{BA} \| r_B \| T_2)$, $Auth_B = h(ID_A \| ID_B \| M_B \| K_{BA} \| r_B \| T_2)$, $z_B = u_B + Auth_B \cdot \sigma_B \pmod{q}$, $r_{B_{mask}} = r_B \oplus h(K_{BA} \| T_2)$, and $SK_B = h(ID_A \| ID_B \| r_B \cdot M_A \| K_{AB} \| K_{BA} \| T_1 \| T_2)$, and sends the message $M_2 = \{PID_B, M_B, T_B, z_B, r_{B_{mask}}, T_2\}$ to U_A over a public channel.
- Step 3: After receiving M_2 from U_B , U_A checks the timestamp $|T_{now} - T_2| \leq \Delta T$. If the timestamp is valid, U_A computes $K_{BA} = h(x_A \cdot M_B \| T_2)$, $r_B = r_{B_{mask}} \oplus h(K_{BA} \| T_2)$, $ID_B = PID_B \oplus h(K_{BA} \| r_B \| T_2)$, and $Auth_B^* = h(ID_A \| ID_B \| M_B \| K_{BA} \| r_B \| T_2)$, and uses $z_B \cdot g \stackrel{?}{=} T_B + Auth_B^* \cdot W_B$ to verify that message M_2 came from user U_B . Upon successful verification, U_A computes the session key $SK_A = h(ID_A \| ID_B \| r_B \cdot M_A \| K_{AB} \| K_{BA} \| T_1 \| T_2)$. Both U_A and U_B share the same session key $SK_A = SK_B$.

6.4. Re-Authentication Phase

Unlike the existing schemes such as Khan et al., whose key update procedure follows a process similar to the authentication phase [19], effectively re-running substantial parts of the AKA procedure, the proposed protocol employs a lightweight re-authentication mechanism that renews the session key using computationally inexpensive operations. As a result, the proposed scheme significantly reduces computational overhead by avoiding costly elliptic curve scalar multiplication during the re-authentication phase. The detailed steps are illustrated in Figure 5.

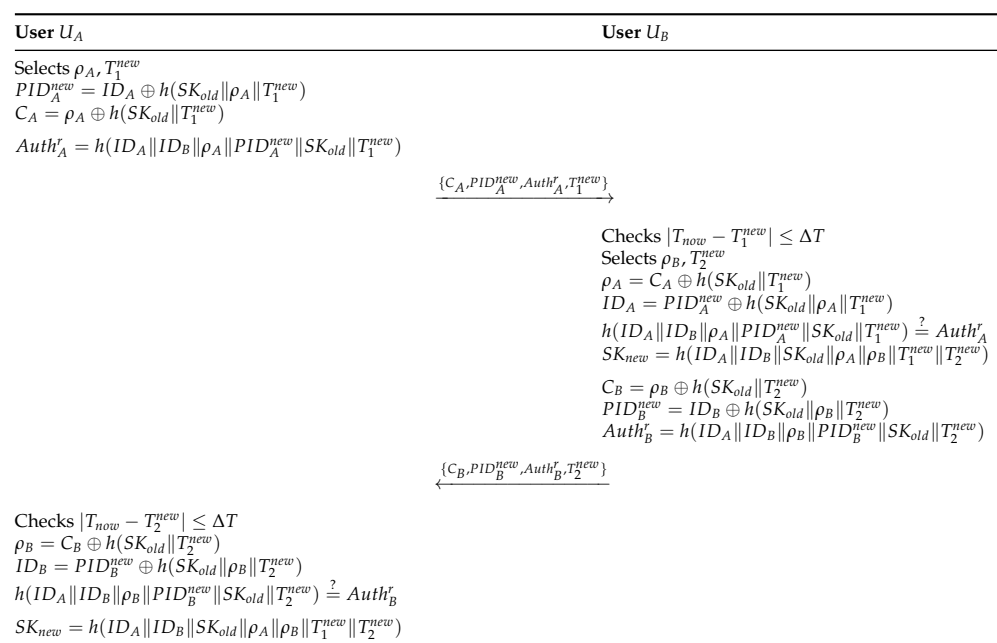


Figure 5. Re-authentication Phase.

- Step 1: U_A selects a random nonce ρ_A and timestamp T_1^{new} , computes $PID_A^{new} = ID_A \oplus h(SK_{old} \parallel \rho_A \parallel T_1^{new})$, $C_A = \rho_A \oplus h(SK_{old} \parallel T_1^{new})$, and $Auth_A^r = h(ID_A \parallel ID_B \parallel \rho_A \parallel PID_A^{new} \parallel SK_{old} \parallel T_1^{new})$, and sends $\{C_A, PID_A^{new}, Auth_A^r, T_1^{new}\}$ to U_B over a public channel.
- Step 2: After receiving the message from U_A , U_B checks the timestamp $|T_{now} - T_1^{new}| \leq \Delta T$. If the timestamp is valid, U_B selects a random nonce ρ_B and timestamp T_2^{new} , computes $\rho_A = C_A \oplus h(SK_{old} \parallel T_1^{new})$ and $ID_A = PID_A^{new} \oplus h(SK_{old} \parallel \rho_A \parallel T_1^{new})$, and uses $h(ID_A \parallel ID_B \parallel \rho_A \parallel PID_A^{new} \parallel SK_{old} \parallel T_1^{new}) \stackrel{?}{=} Auth_A^r$ to verify that the message came from U_A . U_B then computes $SK_{new} = h(ID_A \parallel ID_B \parallel SK_{old} \parallel \rho_A \parallel \rho_B \parallel T_1^{new} \parallel T_2^{new})$, $C_B = \rho_B \oplus h(SK_{old} \parallel T_2^{new})$, $PID_B^{new} = ID_B \oplus h(SK_{old} \parallel \rho_B \parallel T_2^{new})$, and $Auth_B^r = h(ID_A \parallel ID_B \parallel \rho_B \parallel PID_B^{new} \parallel SK_{old} \parallel T_2^{new})$, and sends $\{C_B, PID_B^{new}, Auth_B^r, T_2^{new}\}$ to U_A over a public channel.
- Step 3: After receiving the message from U_B , U_A checks the timestamp $|T_{now} - T_2^{new}| \leq \Delta T$. If the timestamp is valid, U_A computes $\rho_B = C_B \oplus h(SK_{old} \parallel T_2^{new})$ and $ID_B = PID_B^{new} \oplus h(SK_{old} \parallel \rho_B \parallel T_2^{new})$, and uses $h(ID_A \parallel ID_B \parallel \rho_B \parallel PID_B^{new} \parallel SK_{old} \parallel T_2^{new}) \stackrel{?}{=} Auth_B^r$ to verify that the message came from U_B . Upon successful verification, U_A computes the renewed session key $SK_{new} = h(ID_A \parallel ID_B \parallel SK_{old} \parallel \rho_A \parallel \rho_B \parallel T_1^{new} \parallel T_2^{new})$. Both U_A and U_B now share the renewed session key SK_{new} .

7. Security Analysis

7.1. Formal Analysis Using AVISPA

To further validate the robustness of the proposed authentication protocol, formal security verification is conducted using the AVISPA (Automated Validation of Internet Security Protocols and Applications) tool [30]. AVISPA is a well-established framework for the automated analysis of cryptographic protocols and is widely utilized to examine whether security protocols satisfy their intended authentication and confidentiality requirements under adversarial environments. The platform supports systematic exploration of potential protocol weaknesses against a variety of attack models. The proposed protocol is specified using the High-Level Protocol Specification Language (HLPSP), which enables formal modeling of protocol entities, communication procedures, and security objectives. Through HLPSP, authentication and secrecy goals can be explicitly represented and analyzed in a mathematically rigorous manner. The HLPSP description is subsequently translated into the Intermediate Format (IF) by the HLPSP2IF translator, allowing the protocol to be evaluated by multiple automated verification backends. To obtain a comprehensive assessment, the protocol is analyzed using several backend verification engines supported by AVISPA, including the On-the-Fly Model Checker (OFMC), the Constraint-Logic-based Attack Searcher (CL-AtSE), the SAT-based Model Checker (SATMC), and the Tree Automata-based Protocol Analyzer (TA4SP). Since each backend adopts different search strategies and verification methodologies, the combined analysis improves the reliability and completeness of the security evaluation.

For the experimental verification, simulations are carried out using the OFMC and CL-AtSE backends. As illustrated in Figure 6, both analyzers return a *SAFE* status, indicating that no attack violating the specified authentication and secrecy goals is found under the considered Dolev–Yao adversarial model. For OFMC, the protocol analysis explores 216 nodes with a search depth of 9 plies, where the depth represents the number of message-exchange steps considered during state-space exploration, and completes in 0.54 s. For CL-AtSE, 68 protocol states are analyzed and no attack state is found. These statistics indicate that the protocol is systematically explored within the considered state space without revealing any feasible attack path. Therefore, the AVISPA results confirm that the proposed protocol satisfies the desired security goals and can successfully withstand

several well-known network attacks, including replay attacks, impersonation attempts, and man-in-the-middle attacks.

	SUMMARY SAFE
% OFMC % Version of 2006/02/13 SUMMARY SAFE DETAILS BOUNDED_NUMBER_OF_SESSIONS PROTOCOL /home/span/span/testsuite/results/SG.if GOAL as_specified BACKEND OFMC COMMENTS STATISTICS parseTime: 0.00s searchTime: 0.54s visitedNodes: 216 nodes depth: 9 plies	DETAILS BOUNDED_NUMBER_OF_SESSIONS TYPED_MODEL PROTOCOL /home/span/span/testsuite/results/SG.if GOAL As Specified BACKEND CL-AtSe STATISTICS Analysed : 68 states Reachable : 0 states Translation: 0.10 seconds Computation: 0.00 seconds
(a) OFMC	(b) CL-AtSe

Figure 6. Result of AVISPA simulation.

7.2. Formal Analysis Using the RoR Model

To evaluate the session key security of the proposed authentication protocol, this paper employs the Real-or-Random (RoR) security model [31]. The RoR model provides a formal framework for analyzing whether an adversary can successfully distinguish a legitimately established session key from a randomly selected value. By adopting this model, the resistance of the proposed protocol against session key disclosure attacks can be systematically examined. In the considered framework, each communicating party is modeled as an instance denoted by p^t , where p identifies the protocol entity and t specifies the corresponding session instance. During the security experiment, the adversary A is allowed to interact with protocol participants through a set of oracle queries in order to gain knowledge about the generated session key. Moreover, the adversary is assumed to possess full control over the public communication channel, enabling both passive attacks, including message interception, and active attacks, such as replay, injection, and modification of transmitted messages. The detailed query capabilities available to the adversary in the RoR model are summarized in Table 2. Furthermore, $|Hash|$ represents the cardinality of the output space of the hash function $H(\cdot)$, while q_h denotes the maximum number of hash queries that adversary A can perform during the security game.

Table 2. Queries and description in RoR model.

Query	Description
$Execute(p_{U_A}^{t_1}, p_{U_B}^{t_2})$	Simulates passive eavesdropping, where A collects all exchanged messages during the protocol run.
$CorruptDevice(p_U)$	Grants A the ability to obtain information stored in the user device through power analysis, including parameters such as V_i, N_i, A_i, B_i, X_i .
$Send(p^t, M)$	Models active attacks where A sends forged messages M to any participant instance and observes the output.
$Test(\Pi^*)$	Defines a one-time challenge to the adversary. If $c = 1$, the real session key SK is returned; otherwise, a random string of equal length is returned. The adversary A must determine the value of c .

Proof. Let $Adv(A)$ denote the advantage of adversary A in distinguishing the session key. The security is analyzed through a sequence of games from G_0 to G_3 :

- G_0 : This is the initial game in which A has no information. Therefore,

$$Adv(A) = Adv_{G_0}(A) \quad (6)$$

- G_1 : A can issue Execute queries and obtain the messages transmitted over the public channel. However, the transmitted messages $\{PID_A, M_A, z_A, T_A, r_{A_{mask}}, T_1\}$ and $\{PID_B, M_B, T_B, z_B, r_{B_{mask}}, T_2\}$ do not provide any advantage for computing the session key SK . In other words, A cannot derive SK through passive eavesdropping alone. Thus,

$$Adv_{G_0}(A) = Adv_{G_1}(A) \quad (7)$$

- G_2 : A is allowed to perform Send and Hash queries. The only way to compromise SK is to find a hash collision in $h(\cdot)$. By the birthday bound,

$$|Adv_{G_2}(A) - Adv_{G_1}(A)| \leq \frac{q_h^2}{2 |Hash|} \quad (8)$$

- G_3 : A may perform $CorruptDevice(p_U)$ to obtain the user's secret key x_i used in generating SK . However, this value is masked by the identity and password of U_i , and thus A cannot obtain valid information or impersonate U_i . Therefore,

$$|Adv_{G_3}(A) - Adv_{G_2}(A)| \leq \frac{q_s}{|ID_i| |PW_i|} \quad (9)$$

By the triangle inequality,

$$\begin{aligned} |Adv_{G_3}(A) - Adv_{G_1}(A)| &\leq |Adv_{G_3}(A) - Adv_{G_2}(A)| \\ &\quad + |Adv_{G_2}(A) - Adv_{G_1}(A)| \\ &\leq \frac{q_s}{|ID_i| |PW_i|} + \frac{q_h^2}{2 |Hash|} \end{aligned} \quad (10)$$

Hence, the total advantage of the adversary is bounded as follows:

$$Adv(A) \leq \frac{q_s}{|ID_i| |PW_i|} + \frac{q_h^2}{2 |Hash|} \quad (11)$$

Therefore, the adversary's advantage is negligible, and the proposed scheme can be said to achieve semantic security. $\square$

7.3. Formal Analysis Using the BAN Logic

To demonstrate the reliability and security properties of the proposed authentication scheme, the Burrows–Abadi–Needham (BAN) logic [32] is adopted as a formal verification tool. BAN logic provides a systematic framework for evaluating whether communicating entities can authenticate one another and whether the negotiated session key is securely shared between legitimate participants. By utilizing this logic, the mutual authentication process and the correctness of session key establishment can be formally validated. The BAN logic analysis proceeds through several stages. Initially, the desired authentication and session key agreement objectives of the protocol are specified. Subsequently, the original protocol messages exchanged during communication are converted into idealized expressions compatible with BAN logic notation. Afterward, a set of initial assumptions describing the beliefs, freshness conditions, and trust relationships among the protocol

entities is introduced. Based on these assumptions, BAN logic inference rules and postulates are applied to determine whether the communicating parties can ultimately derive the intended authentication goals and confirm the legitimacy of the shared session key. For clarity, the BAN logic symbols and expressions employed throughout the verification process are first defined together with their corresponding meanings. The principal BAN logic notations used in this paper are summarized in Table 3.

Table 3. BAN logic notations.

Notation	Description
ρ_1, ρ_2	Two principals
s_1, s_2	Two statements
$\rho_1 \mid \equiv s_1$	ρ_1 believes s_1
$\rho_1 \mid \sim s_1$	ρ_1 once said s_1
$\rho_1 \Rightarrow s_1$	ρ_1 controls s_1
$\rho_1 \triangleleft \mu_1$	ρ_1 receives s_1
$\#s_1$	s_1 is fresh
$(s_1)_K$	s_1 is encrypted with K
$\rho_1 \xleftrightarrow{K} \rho_2$	ρ_1 and ρ_2 have shared key K
SK	The session key

7.3.1. BAN Logic Rules

1. Message meaning rule (MMR):

$$\frac{\rho_1 \mid \equiv \rho_1 \xleftrightarrow{K} \rho_2, \quad \rho_1 \triangleleft (s_1)_K}{\rho_1 \mid \equiv \rho_2 \mid \sim s_1}$$

2. Nonce verification rule (NVR):

$$\frac{\rho_1 \mid \equiv \#(s_1), \quad \rho_1 \mid \equiv \rho_2 \mid \sim s_1}{\rho_1 \mid \equiv \rho_2 \mid \equiv s_1}$$

3. Jurisdiction rule (JR):

$$\frac{\rho_1 \mid \equiv \rho_2 \mid \Rightarrow s_1, \quad \rho_1 \mid \equiv \rho_2 \mid \equiv s_1}{\rho_1 \mid \equiv s_1}$$

4. Belief rule (BR):

$$\frac{\rho_1 \mid \equiv (s_1, s_2)}{\rho_1 \mid \equiv s_1}$$

5. Freshness rule (FR):

$$\frac{\rho_1 \mid \equiv \#(s_1)}{\rho_1 \mid \equiv \#(s_1, s_2)}$$

7.3.2. Goals

The authentication goals are to establish that both the U_A and the U_B believe they share a common session key SK , and that each believes that the other also believes so.

Goal 1: $U_A \mid \equiv U_A \xleftrightarrow{SK} U_B$

Goal 2: $U_B \mid \equiv U_A \xleftrightarrow{SK} U_B$

Goal 3: $U_A \mid \equiv U_B \mid \equiv U_A \xleftrightarrow{SK} U_B$

Goal 4: $U_B \mid \equiv U_A \mid \equiv U_A \xleftrightarrow{SK} U_B$

7.3.3. Idealized Forms

To perform the BAN logic analysis, we convert the transmitted messages of the proposed protocol into idealized forms. The idealized forms of the proposed scheme are as follows:

MSG1: $U_A \rightarrow U_B : \{ID_A, r_A, M_A, T_1\}_{K_{AB}}$

MSG2: $U_B \rightarrow U_A : \{ID_B, r_B, M_B, T_2\}_{K_{BA}}$

7.3.4. Assumptions

A1: $U_B \models \#(T_1)$

A2: $U_A \models \#(T_2)$

A3: $U_A \models U_A \xleftrightarrow{K_{AB}} U_B$

A4: $U_B \models U_B \xleftrightarrow{K_{AB}} U_A$

A5: $U_A \models U_A \xleftrightarrow{K_{BA}} U_B$

A6: $U_B \models U_B \xleftrightarrow{K_{BA}} U_A$

A7: $U_A \models U_B \Rightarrow (U_A \xleftrightarrow{SK} U_B)$

A8: $U_B \models U_A \Rightarrow (U_A \xleftrightarrow{SK} U_B)$

7.3.5. BAN Logic Proof

Proof. The process of analysis using BAN logic includes the detailed steps presented below.

Step 1: According Msg_1 , S_1 is derived.

$$S_1 : U_B \triangleleft \{ID_A, r_A, M_A, T_1\}_{K_{AB}}$$

Step 2: Applying S_1 and A_4 to the MMR, S_2 is obtained.

$$S_2 : U_B \models U_A \mid \sim (ID_A, r_A, M_A, T_1)$$

Step 3: Applying A_1 to the FR, S_3 is obtained.

$$S_3 : U_B \models \#(ID_A, r_A, M_A, T_1)$$

Step 4: Applying S_2 and S_3 to the NVR, S_4 is obtained.

$$S_4 : U_B \models U_A \models (ID_A, r_A, M_A, T_1)$$

Step 5: According Msg_2 , S_5 is derived.

$$S_5 : U_A \triangleleft \{ID_B, r_B, M_B, T_2\}_{K_{BA}}$$

Step 6: Applying S_5 and A_5 to the MMR, S_6 is obtained.

$$S_6 : U_A \models U_B \mid \sim (ID_B, r_B, M_B, T_2)$$

Step 7: Applying A_2 to the FR, S_7 is obtained.

$$S_7 : U_A \models \#(ID_B, r_B, M_B, T_2)$$

Step 8: Applying S_6 and S_7 to the NVR, S_8 is obtained.

$$S_8 : U_A \models U_B \models (ID_B, r_B, M_B, T_2)$$

Step 9: M_i and D_j believe that the other party can compute the session key $SK_B = h(ID_A \parallel ID_B \parallel r_B \cdot M_A \parallel K_{AB} \parallel K_{BA} \parallel T_1 \parallel T_2)$, because SK can be calculated using the shared values.

$$S_9 : U_A \models U_B \models U_A \xleftrightarrow{SK} U_B \text{ (Goal 3)}$$

$$S_{10} : U_B \models U_A \models U_A \xleftrightarrow{SK} U_B \text{ (Goal 4)}$$

Step 10: Applying S_9 and A_7 to the JR, S_{11} is obtained. Furthermore, applying S_{10} and A_8 to the JR, S_{12} is obtained.

$$S_{11} : M_i \mid \equiv M_i \xleftrightarrow{SK} D_j \textbf{(Goal 1)}$$

$$S_{12} : D_j \mid \equiv M_i \xleftrightarrow{SK} D_j \textbf{(Goal 2)}$$

□

7.4. Informal Security Analysis

This section explains how the proposed protocol preserves essential security properties and defends against various security attacks. The following provides a comprehensive overview of the informal security analysis.

7.4.1. Impersonation Attack

A malicious adversary $\mathcal{A}$ may attempt to impersonate a legitimate user U_A by generating a forged authentication message. However, to successfully construct a valid login request message $M_1 = \{PID_A, M_A, z_A, T_A, r_A^{mask}, T_1\}$, the attacker must compute a valid signature value z_A , which requires knowledge of the secret parameter σ_A . Since σ_A is derived from the trusted authority's secret key and the user's private credentials, it is not available to the attacker. Without the correct value of σ_A , the attacker cannot satisfy the verification equation $z_A \cdot g = T_A + Auth_A \cdot W_A$. Therefore, the receiver will reject the forged message, and the proposed protocol is secure against impersonation attacks.

7.4.2. ESL Attack

In this attack scenario, the adversary $\mathcal{A}$ is assumed to obtain one ephemeral secret value. The session key is derived as $SK = h(ID_A \| ID_B \| r_B \cdot M_A \| K_{AB} \| K_{BA} \| T_1 \| T_2)$, where $K_{AB} = h(r_A \cdot X_B \| T_1)$ and $K_{BA} = h(r_B \cdot X_A \| T_2)$. If r_A is leaked, the adversary may compute K_{AB} , but cannot derive K_{BA} without knowledge of r_B . Similarly, if r_B is leaked, the adversary may compute K_{BA} , but K_{AB} remains protected by r_A . Moreover, reconstructing the shared point $r_B \cdot M_A = r_B \cdot r_A \cdot g$ requires both ephemeral secrets. Therefore, the leakage of a single ephemeral secret is insufficient to reconstruct the session key, and the proposed protocol improves resistance against ELS attacks.

7.4.3. Data Confidentiality

Data confidentiality ensures that transmitted messages cannot be understood by unauthorized entities. In the proposed protocol, sensitive information is protected using cryptographic operations derived from secret parameters and fresh random numbers. Even if an adversary intercepts transmitted messages, the absence of the required secret values prevents the recovery of plaintext information. Therefore, the proposed protocol preserves data confidentiality.

7.4.4. Anonymity

User identity privacy is maintained through a dynamic pseudonym mechanism, where the real identity ID_A is protected using a value generated as $PID_A = ID_A \oplus h(K_{AB} \| r_A \| T_1)$. Since the pseudonym is generated using fresh random numbers and timestamps, it changes in each session. Without knowledge of the secret parameters, the adversary cannot recover the real identity from the transmitted pseudonym. Thus, the proposed protocol provides user anonymity.

7.4.5. Untraceability

In the proposed protocol, dynamic pseudonyms and fresh random numbers are used in every session. As a result, the transmitted values differ across sessions even for the same

user. Therefore, the adversary cannot correlate different communication sessions to the same entity, and the proposed protocol ensures untraceability.

7.4.6. Message Authentication

Message authentication ensures that received messages originate from legitimate participants and have not been modified during transmission. In the proposed protocol, each message includes authentication values generated using secret parameters and session-specific data. Any modification of transmitted messages will cause verification failure at the receiving entity. Therefore, the proposed protocol guarantees message authentication and message integrity.

7.4.7. DoS Attack

To mitigate DoS attacks, each participant performs lightweight verification operations, such as timestamp validation and hash comparison, before executing computationally expensive operations. Invalid requests are discarded at an early stage, thereby reducing unnecessary computational overhead caused by malicious requests. Accordingly, the proposed protocol is resilient against unnecessary computation caused by DoS attacks.

7.4.8. Replay Attack

Replay attacks occur when an adversary retransmits previously intercepted messages to gain unauthorized access. In the proposed protocol, each message carries a timestamp value, such as T_1 and T_2 , to ensure message freshness. Any outdated message is detected through timestamp verification and immediately rejected. Thus, the proposed protocol is secure against replay attacks.

7.4.9. Privileged Insider Attack

A privileged insider may attempt to misuse stored system information to impersonate legitimate users or derive secret credentials. In the proposed protocol, sensitive parameters stored in the system are protected using one-way hash functions and cryptographic masking techniques. Even if an insider gains access to stored data, the absence of required secret parameters prevents the derivation of authentication credentials or session keys. Hence, the proposed protocol is secure against privileged insider attacks.

7.4.10. Stolen Device Attack

When a device is stolen, an adversary may attempt to extract stored information to impersonate the legitimate user. In the proposed protocol, values stored in the device are protected using hash functions and secret-dependent computations. Without knowledge of the user's password or secret parameters, the adversary cannot generate valid authentication messages. Therefore, the proposed protocol is secure against stolen device attacks.

7.4.11. Perfect Forward Secrecy

PFS ensures that compromise of long-term secrets does not lead to disclosure of previously established session keys. In the proposed protocol, the session key is established as $SK = h(ID_A \parallel ID_B \parallel r_B \cdot M_A \parallel K_{AB} \parallel K_{BA} \parallel T_1 \parallel T_2)$ where $M_A = r_A \cdot g$ and $M_B = r_B \cdot g$ are generated using fresh random nonces r_A and r_B for each session. Even if an adversary compromises long-term secrets such as x_A , x_B , or σ_i , recovering previously established session keys still requires reconstruction of the ephemeral Diffie–Hellman shared secret $r_A r_B g$. However, deriving r_A or r_B from the publicly transmitted values M_A and M_B is computationally infeasible due to the hardness of the ECDLP. Therefore, compromise of long-term secrets does not enable an adversary to recover previously established session keys, and the proposed protocol provides PFS.

7.4.12. Long-Term Key Compromise Attack

Assume that an adversary compromises the long-term private key of user U_A . Even with knowledge of the long-term secret, the adversary still cannot impersonate another legitimate user U_B to establish a valid session with U_A . In the proposed protocol, successful authentication requires generation of valid authentication parameters, such as $z_B = u_B + Auth_B \cdot \sigma_B \pmod{q}$ where σ_B is derived from the secret values of U_B , and u_B and r_B are freshly generated random nonces for each session. Since these session-specific values are not directly transmitted and cannot be reconstructed from intercepted messages, the adversary cannot forge a valid authentication message on behalf of U_B . Furthermore, the session key additionally depends on the ephemeral Diffie–Hellman shared secret $r_B \cdot M_A = r_A r_B g$ which cannot be reconstructed without knowledge of the fresh random nonces. Therefore, compromise of one party's long-term private key does not enable successful KCI attacks or recovery of the session key in the proposed protocol.

7.5. Security Features Comparison

We compared the security features of the proposed protocol with six representative authentication schemes, including Khan et al. [19], Nkenyereye et al. [26], Alzahrani et al. [33], Shariq et al. [34], Peivandizadeh et al. [35], and Wu et al. [18]. Although [33,34] were designed for different environments, such as IoT D2D systems and drone communications, they have similar security properties and operational characteristics with the proposed protocol, including mutual authentication, session key agreement, anonymity, untraceability, and lightweight operation. Therefore, these schemes are considered relevant for comparative security and performance analysis. The comparison results are summarized in Table 4.

Table 4. Security Feature Comparison.

Security Features	Proposed	Khan [19]	Nkenyereye [26]	Alzahrani [33]	Shariq [34]	Peivandizadeh [35]	Wu [18]
Impersonation Attack	✓	×	✓	×	✓	✓	✓
ESL Attack	✓	×	✓	✓	✓	×	✓
Replay Attack	✓	✓	✓	✓	✓	✓	✓
DoS Attack	✓	×	✓	×	✓	✓	×
Privileged Insider Attack	✓	×	✓	×	×	×	×
Stolen Device Attack	✓	×	×	✓	✓	×	✓
Data Confidentiality	✓	×	×	✓	×	✓	✓
Anonymity	✓	×	×	✓	✓	×	✓
Untraceability	✓	×	×	✓	✓	×	✓
Message Authentication	✓	×	×	✓	✓	✓	✓
Perfect Forward Secrecy	✓	×	×	✓	✓	✓	✓
KCI Attack	✓	×	×	×	×	×	✓

Note: (✓) Secure against the attack or supports the feature, (×) Insecure against the attack or lacks the feature.

The results show that the proposed protocol satisfies all considered security requirements, whereas some existing schemes fail to provide resistance against specific attacks. In particular, as demonstrated in Section 5, Khan et al.'s scheme is vulnerable to multiple security threats, including impersonation, ESL, data confidentiality violation, anonymity and untraceability breaches, message authentication failure, and DoS attacks.

7.6. Communication Cost Comparison

In this section, we evaluate the communication cost of the proposed scheme and compare it with several existing schemes, including Khan et al. [19], Nkenyereye et al. [26], Alzahrani et al. [33], Shariq et al. [34], Peivandizadeh et al. [35], and Wu et al. [18].

To ensure a fair comparison, all schemes are evaluated under the same parameter-size assumptions. Specifically, the sizes of the hash output, identity (or pseudonym), and random nonce are set to 160 bits each, while the timestamp and elliptic curve point

are set to 32 bits and 320 bits, respectively. The 160-bit size for the identity, random nonce, and hash output was selected based on commonly adopted parameter assumptions in the smart grid authentication literature, where SHA-1-based hash settings are frequently used for communication cost analysis. In addition, the 32-bit timestamp assumption is adopted for freshness verification, while the 320-bit elliptic curve point size follows commonly used ECC parameter settings, where ECC security is maintained at the 160-bit level [35].

In addition, the size of the ciphertext is determined based on the total bit-length of the encapsulated plaintext parameters.

The communication cost is computed by summing the bit-lengths of all transmitted parameters during the authentication and key agreement phase. The results are summarized in Table 5.

Table 5. Comparison analysis of communication costs.

Scheme	Total Cost (Bits)	Number of Messages
Proposed	2304 bits	2
Khan [19]	1184 bits	2
Nkenyereye [26]	1760 bits	2
Alzahrani [33]	2304 bits	2
Shariq [34]	1664 bits	2
Peivandizadeh [35]	1334 bits	2
Wu [18]	1536 bits	3

As shown in Table 5, the proposed scheme incurs a relatively higher communication cost compared to some existing schemes. This is primarily due to the inclusion of additional authentication parameters to enhance the security of the protocol. In contrast, some existing schemes achieve lower communication costs by adopting simplified constructions or by encapsulating multiple parameters within encrypted messages. However, such approaches may compromise the overall robustness of the authentication mechanism. Therefore, the increased communication overhead of the proposed scheme can be regarded as a reasonable trade-off for achieving improved security properties.

7.7. Computation Costs Comparison

In this section, we evaluate the computational cost of the proposed protocol during the AKA phase and compare it with several related schemes [18,19,26,33–35]. To ensure a fair comparison, all schemes are analyzed under identical assumptions, and lightweight operations such as bitwise XOR and concatenation are excluded due to their negligible computational overhead. The considered cryptographic operations include a one-way hash function (T_h), elliptic curve scalar multiplication (T_{ecpm}), symmetric encryption/decryption (T_{se}/T_{sd}), and HMAC computation (T_{hmac}).

The experiments were conducted on a macOS Sequoia 15.5 system equipped with an Apple M4 Pro processor and 24 GB of memory, utilizing the OpenSSL cryptographic library. The average execution time of each cryptographic primitive is summarized in Table 6, while the computational cost in terms of the number of operations is presented in Table 7. The reported computational time was evaluated on a general-purpose laptop environment for comparative analysis purposes. Therefore, the measured execution times may differ from actual performance in practical smart grid or edge computing environments with resource-constrained devices. Based on these measurements, the estimated execution times for each scheme are reported in Table 8.

Table 6. Average execution time of cryptographic operations.

Operation	Symbol	Time (ms)
Hash function	T_h	0.000376
ECC point multiplication	T_{ecpm}	0.004714
Symmetric encryption	T_{se}	0.000318
Symmetric decryption	T_{sd}	0.000277
HMAC operation	T_{hmac}	0.000697

Table 7. Comparison analysis of computation costs.

Scheme	Environment	Entity 1	Entity 2	Total
Khan [19]	U2U	$2T_{ecpm} + 7T_h + T_{se} + T_{sd}$	$2T_{ecpm} + 6T_h + T_{se} + T_{sd}$	$4T_{ecpm} + 13T_h + 2T_{se} + 2T_{sd}$
Nkenyereye [26]	U2U	$4T_{ecpm} + 5T_h$	$4T_{ecpm} + 4T_h$	$8T_{ecpm} + 9T_h$
Alzahrani [33]	D2D	$2T_{ecpm} + 4T_h$	$2T_{ecpm} + 3T_h$	$4T_{ecpm} + 7T_h$
Shariq [34]	D2D	$4T_{ecpm} + 6T_h + 2T_{hmac}$	$4T_{ecpm} + 6T_h + 2T_{hmac}$	$8T_{ecpm} + 12T_h + 4T_{hmac}$
Peivandizadeh [35]	SM-SP	$2T_{ecpm} + 6T_h + T_{se} + T_{sd}$	$2T_{ecpm} + 6T_h + T_{se} + T_{sd}$	$4T_{ecpm} + 12T_h + 2T_{se} + 2T_{sd}$
Wu [18]	SM-SP	$4T_{ecpm} + 6T_h + 2T_{hmac}$	$4T_{ecpm} + 6T_h + 2T_{hmac}$	$8T_{ecpm} + 12T_h + 4T_{hmac}$
Proposed	U2U	$6T_{ecpm} + 12T_h$	$6T_{ecpm} + 12T_h$	$12T_{ecpm} + 24T_h$

Table 8. Comparison of Computation Costs (Estimated Execution Time).

Schemes	Entity 1 (ms)	Entity 2 (ms)	Total Time (ms)
Khan [19]	≈ 0.0127	≈ 0.0123	≈ 0.0250
Nkenyereye [26]	≈ 0.0207	≈ 0.0204	≈ 0.0411
Alzahrani [33]	≈ 0.0109	≈ 0.0106	≈ 0.0215
Shariq [34]	≈ 0.0225	≈ 0.0225	≈ 0.0450
Peivandizadeh [35]	≈ 0.0123	≈ 0.0123	≈ 0.0246
Wu [18]	≈ 0.0225	≈ 0.0225	≈ 0.0450
Proposed	≈ 0.0328	≈ 0.0328	≈ 0.0656

As shown in Table 8, the proposed scheme exhibits higher computational cost compared to several existing approaches. As indicated in Table 5, the proposed protocol requires additional ECC scalar multiplication operations compared to related schemes, mainly due to the cryptographic verification procedures introduced for decentralized U2U authentication.

Unlike centralized SG architectures, decentralized U2U environments cannot rely on a trusted authority during the authentication phase. Therefore, communicating users must independently perform mutual authentication, session key validation, and cryptographic verification over public channels. In particular, the proposed protocol employs additional ECC-based verification operations inspired by Schnorr-style authentication mechanisms to securely bind session-specific parameters with long-term credentials in decentralized communication environments. These verification operations also prevent the direct exposure of long-term credentials by securely associating them with session-specific authentication parameters during the authentication process. For example, some existing schemes achieve lower computational cost with fewer cryptographic operations; however, they may not fully provide resistance against impersonation attacks, privileged insider attacks, and ESL attacks. In contrast, the proposed protocol is designed to strengthen mutual authentication and improve resistance against impersonation, privileged insider, and ESL attacks in decentralized U2U communication environments.

Although the proposed protocol incurs relatively higher computational overhead, the additional ECC operations contribute to stronger authentication guarantees and secure session key establishment without requiring the direct involvement of a trusted authority

during the authentication phase. Therefore, the proposed protocol remains applicable to secure decentralized U2U communication environments in smart grids.

7.8. Communication Latency

This subsection provides an analytical estimate of the authentication latency of the proposed U2U authentication phase by combining the communication overhead in Table 5 with the computation time in Table 8. This estimate considers only payload transmission and cryptographic computation; network-dependent factors such as MAC/PHY headers, propagation delay, queuing delay, channel contention, and retransmissions are not included. The proposed protocol exchanges two messages between users U_A and U_B , and its total payload size is 2304 bits (288 bytes). Therefore, following the standard transmission delay model [36], when R denotes the data rate in bits per second, the payload transmission delay in milliseconds is calculated as

$$T_{tx} = \frac{2304}{R} \times 10^3 \text{ ms.} \quad (12)$$

The estimated authentication latency is then obtained as

$$T_{E2E}^{est} \approx T_{tx} + T_{comp}, \quad (13)$$

where T_{comp} denotes the total computation time required by both users. Based on Table 8, the total computation time of the proposed protocol is approximately 0.0656 ms. For example, at an illustrative data rate of 50 Mbps,

$$T_{tx} = \frac{2304}{50 \times 10^6} \times 10^3 \approx 0.04608 \text{ ms,} \quad (14)$$

$$T_{E2E}^{est} \approx 0.04608 + 0.0656 = 0.11168 \text{ ms.} \quad (15)$$

Under the same analytical model at the payload level, the estimated latency is approximately 0.2960 ms at 10 Mbps, 0.4496 ms at 6 Mbps, 0.8336 ms at 3 Mbps, and 2.3696 ms at 1 Mbps. These results should be interpreted as comparative analytical estimates rather than measured end-to-end network latency in deployed smart grid systems. Nevertheless, they indicate that the two-message structure of the proposed U2U authentication protocol limits the communication delay introduced by the additional security parameters.

7.9. Estimated Energy Consumption

Because the present evaluation does not assume a specific smart meter, gateway, or user device hardware platform, this subsection reports a parametric energy estimate rather than a device-specific measurement. Let P_{pair} denote the aggregate average active power consumption of the two participating user devices, including their communication interfaces, during one authentication execution. Using the latency estimate derived above and the standard energy model $E = P \times T$ [37], the authentication energy can be expressed as

$$E_{auth}^{est} = P_{pair} \times T_{E2E}^{est}, \quad (16)$$

where P_{pair} is expressed in watts and T_{E2E}^{est} is expressed in seconds. At 50 Mbps, $T_{E2E}^{est} \approx 0.11168 \text{ ms}$; therefore,

$$E_{auth}^{est} \approx P_{pair} \times 0.11168 \times 10^{-3} \text{ J} = 0.11168 P_{pair} \text{ mJ.} \quad (17)$$

For cryptographic computation only, Table 8 reports approximately 0.0328 ms for each user. If the two user devices have average active computation powers P_A and P_B , respectively, the computation energy is estimated as

$$E_{comp}^{est} \approx (0.0328P_A + 0.0328P_B) \text{ mJ}. \quad (18)$$

If both users are implemented on devices with the same active computation power P_{dev} , this becomes $E_{comp}^{est} \approx 0.0656P_{dev} \text{ mJ}$. These values scale linearly with the assumed device power and should therefore be regarded as platform-dependent estimates. A more precise energy evaluation requires implementation on representative smart grid devices and direct measurement of their active power consumption during authentication.

8. Conclusions

In this paper, we proposed a secure and efficient ECC-based user-to-user authentication and key agreement protocol for smart grid environments. The proposed protocol enables mutual authentication between two users and establishes a secure session key without requiring the involvement of a trusted authority during the authentication phase, making it suitable for fully decentralized U2U communication scenarios.

Through security analysis, the proposed scheme resists various attacks, including impersonation attacks, replay attacks, and privileged insider attacks. In addition, the protocol incorporates pseudonym-based identity protection and session-specific parameter binding to achieve user anonymity and unlinkability across sessions, while ensuring strong resistance against impersonation attacks through robust mutual authentication mechanisms. Furthermore, performance evaluation demonstrates that the proposed protocol remains practical for real-world deployment while providing strong security guarantees. This makes the proposed scheme well suited for applications that require secure decentralized communication. Therefore, the proposed protocol provides a reliable and secure solution for decentralized smart grid communication, where direct user-to-user authentication is required.

As future work, we will consider reducing the computational overhead and extending the proposed protocol to support more complex scenarios, such as multi-user environments and integration with emerging IoT-based smart grid applications. In addition, since the current performance evaluation is primarily based on simulation and experimental analysis under controlled environments, actual performance may differ in practical smart grid deployment scenarios. Therefore, future work will include implementation and evaluation in real-world smart grid environments.

Author Contributions: Conceptualization, Y.N. and Y.P.; Methodology, Y.N.; Validation, Y.P.; Formal analysis, H.P.; Investigation, Y.N.; Writing—original draft, Y.N. and H.P.; Writing—review & editing, H.P. and Y.P.; Supervision, Y.P.; Project administration, Y.P.; Funding acquisition, Y.P. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Fang, X.; Misra, S.; Xue, G.; Yang, D. Smart Grid—The New and Improved Power Grid: A Survey. *IEEE Commun. Surv. Tutor.* **2012**, *14*, 944–980. <https://doi.org/10.1109/SURV.2011.101911.00087>.

2. Yan, Y.; Qian, Y.; Sharif, H.; Tipper, D. A Survey on Cyber Security for Smart Grid Communications. *IEEE Commun. Surv. Tutor.* **2012**, *14*, 998–1010. <https://doi.org/10.1109/SURV.2012.010912.00035>.
3. Mojumder, M.R.H.; Antara, F.A.; Hasanuzzaman, M.; Alamri, B.; Alsharef, M. Electric Vehicle-to-Grid (V2G) Technologies: Impact on the Power Grid and Battery. *Sustainability* **2022**, *14*, 13856. <https://doi.org/10.3390/su142113856>.
4. Tushar, W.; Saha, T.K.; Yuen, C.; Smith, D.; Poor, H.V. Peer-to-peer trading in electricity networks: An overview. *IEEE Trans. Smart Grid* **2020**, *11*, 3185–3200.
5. Son, S.; Lee, J.; Park, Y.; Park, Y.; Das, A.K. Design of blockchain-based lightweight V2I handover authentication protocol for VANET. *IEEE Trans. Netw. Sci. Eng.* **2022**, *9*, 1346–1358.
6. Wazid, M.; Bagga, P.; Das, A.K.; Shetty, S.; Rodrigues, J.J.; Park, Y. AKM-IoV: Authenticated key management protocol in fog computing-based Internet of vehicles deployment. *IEEE Internet Things J.* **2019**, *6*, 8804–8817.
7. Gautam, D.; Thakur, G.; Kumar, P.; Das, A.K.; Park, Y. Blockchain assisted intra-twin and inter-twin authentication scheme for vehicular digital twin system. *IEEE Trans. Intell. Transp. Syst.* **2024**, *25*, 15002–15015.
8. Kwon, D.; Son, S.; Kim, M.; Lee, J.; Das, A.K.; Park, Y. A secure self-certified broadcast authentication protocol for intelligent transportation systems in UAV-assisted mobile edge computing environments. *IEEE Trans. Intell. Transp. Syst.* **2024**, *25*, 19004–19017.
9. Vangala, A.; Das, A.K.; Mitra, A.; Das, S.K.; Park, Y. Blockchain-enabled authenticated key agreement scheme for mobile vehicles-assisted precision agricultural IoT networks. *IEEE Trans. Inf. Forensics Secur.* **2022**, *18*, 904–919.
10. Zibaeirad, A.; Koleini, F.; Bi, S.; Hou, T.; Wang, T. A Comprehensive Survey on the Security of Smart Grid: Challenges, Mitigations, and Future Research Opportunities. *arXiv* **2024**, arXiv:2407.07966.
11. Asghar, M.R.; Dán, G.; Miorandi, D.; Chlamtac, I. Smart Meter Data Privacy: A Survey. *IEEE Commun. Surv. Tutor.* **2017**, *19*, 2820–2835. <https://doi.org/10.1109/COMST.2017.2720195>.
12. Chim, T.W.; Yiu, S.M.; Hui, L.C.K.; Li, V.O.K. PASS: Privacy-Preserving Authentication Scheme for Smart Grid Network. In *Proceedings of the IEEE International Conference on Smart Grid Communications (SmartGridComm)*; IEEE: Piscataway, NJ, USA, 2011; pp. 196–201. <https://doi.org/10.1109/SmartGridComm.2011.6102314>.
13. Nicanfar, H.; Leung, V.C.M. Multilayer Consensus ECC-Based Password Authenticated Key-Exchange (MCEPAK) Protocol for Smart Grid System. *IEEE Trans. Smart Grid* **2013**, *4*, 253–264. <https://doi.org/10.1109/TSG.2012.2226252>.
14. Odelu, V.; Das, A.K.; Wazid, M.; Conti, M. Provably Secure Authenticated Key Agreement Scheme for Smart Grid. *IEEE Trans. Smart Grid* **2018**, *9*, 1900–1910. <https://doi.org/10.1109/TSG.2016.2602282>.
15. Wazid, M.; Das, A.K.; Kumar, N.; Rodrigues, J.J.P.C. Secure Three-Factor User Authentication Scheme for Renewable-Energy-Based Smart Grid Environment. *IEEE Trans. Ind. Inform.* **2017**, *13*, 3144–3153. <https://doi.org/10.1109/TII.2017.2732999>.
16. Gope, P.; Sikdar, B. Privacy-Aware Authenticated Key Agreement Scheme for Secure Smart Grid Communication. *IEEE Trans. Smart Grid* **2019**, *10*, 3953–3961. <https://doi.org/10.1109/TSG.2018.2844403>.
17. Hu, S.; Chen, Y.; Zheng, Y.; Xing, B.; Li, Y.; Zhang, L.; Chen, L. Provably Secure ECC-Based Authentication and Key Agreement Scheme for Advanced Metering Infrastructure in the Smart Grid. *IEEE Trans. Ind. Inform.* **2023**, *19*, 5985–5994. <https://doi.org/10.1109/TII.2022.3191319>.
18. Wu, Y.; Guo, H.; Han, Y.; Li, S.; Liu, J. A Security-Enhanced Authentication and Key Agreement Protocol in Smart Grid. *IEEE Trans. Ind. Inform.* **2024**, *20*, 11449–11457. <https://doi.org/10.1109/TII.2024.3399915>.
19. Khan, A.A.; Kumar, V.; Prasad, R.; Idrisi, M.J. SGAK: A Robust ECC-Based Authenticated Key Exchange Protocol for Smart Grid Networks. *IEEE Access* **2024**, *12*, 195745–195759. <https://doi.org/10.1109/ACCESS.2024.3434532>.
20. Fouda, M.M.; Fadlullah, Z.M.; Kato, N.; Lu, R.; Shen, X. A Lightweight Message Authentication Scheme for Smart Grid Communications. *IEEE Trans. Smart Grid* **2011**, *2*, 675–685. <https://doi.org/10.1109/TSG.2011.2160661>.
21. Tsai, J.L.; Lo, N.W. Secure Anonymous Key Distribution Scheme for Smart Grid. *IEEE Trans. Smart Grid* **2016**, *7*, 906–914. <https://doi.org/10.1109/TSG.2015.2440658>.
22. Abbasinezhad-Mood, D.; Nikooghadam, M. Design and hardware implementation of a security-enhanced elliptic curve cryptography based lightweight authentication scheme for smart grid communications. *Future Gener. Comput. Syst.* **2018**, *84*, 47–57. <https://doi.org/10.1016/j.future.2018.02.034>.
23. Yu, S.; Das, A.K.; Park, Y.; Lorenz, P. SLAP-IoD: Secure and lightweight authentication protocol using physical unclonable functions for internet of drones in smart city environments. *IEEE Trans. Veh. Technol.* **2022**, *71*, 10374–10388.
24. Yu, S.; Park, K.; Park, Y. A machine learning attack-resistant PUF-based robust and efficient mutual authentication scheme in fog-enabled IoT environments. *IEEE Internet Things J.* **2025**, *12*, 20652–20669.
25. Srinivas, J.; Das, A.K.; Li, X.; Khan, M.K.; Jo, M. Designing Anonymous Signature-Based Authenticated Key Exchange Scheme for Internet of Things-Enabled Smart Grid Systems. *IEEE Trans. Ind. Inform.* **2021**, *17*, 4425–4435. <https://doi.org/10.1109/TII.2020.3011849>.

26. Nkenyereye, L.; Thakare, A.; Khataniar, P.; Imandi, R.; B N, P.K. Lightweight Authentication Protocol for Smart Grids: An Energy-Efficient Authentication Scheme for Resource-Limited Smart Meters. *Mathematics* **2025**, *13*, 580. <https://doi.org/10.3390/math13040580>.
27. Hankerson, D.; Menezes, A.; Vanstone, S. *Guide to Elliptic Curve Cryptography*; Springer: New York, NY, USA, 2004.
28. Schnorr, C.P. Efficient Signature Generation by Smart Cards. *J. Cryptol.* **1991**, *4*, 161–174. <https://doi.org/10.1007/BF00196725>.
29. Canetti, R.; Krawczyk, H. Analysis of Key-Exchange Protocols and Their Use for Building Secure Channels. In *Advances in Cryptology—EUROCRYPT 2001*; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2001; Volume 2045, pp. 453–474. https://doi.org/10.1007/3-540-44987-6_28.
30. Armando, A.; Basin, D.; Boichut, Y.; Chevalier, Y.; Compagna, L.; Cuéllar, J.; Drielsma, P.H.; Héam, P.C.; Kouchnarenko, O.; Mantovani, J.; et al. The AVISPA tool for the automated validation of internet security protocols and applications. In *Computer Aided Verification, Proceedings of the 17th International Conference, CAV 2005, Edinburgh, Scotland, UK, 6–10 July 2005*; Springer: Berlin/Heidelberg, Germany, 2005; pp. 281–285.
31. Abdalla, M.; Fouque, P.A.; Pointcheval, D. Password-based authenticated key exchange in the three-party setting. In *Proceedings of the International Workshop on Public Key Cryptography*; Springer: Berlin/Heidelberg, Germany, 2005; pp. 65–84.
32. Burrows, M.; Abadi, M.; Needham, R. A logic of authentication. *ACM Trans. Comput. Syst. (TOCS)* **1990**, *8*, 18–36.
33. Alzahrani, N. A Verifiably Secure and Lightweight Device-to-Device (D2D) Authentication Protocol for the Resource-Constrained IoT Networks. *IEEE Access* **2025**, *13*, 92982–92996. <https://doi.org/10.1109/ACCESS.2025.3568692>.
34. Shariq, M.; Jamil, N.; Rawat, G.S.; Chaudhry, S.A.; Masud, M.; Das, A.K. Provably Secure and Reliable Privacy-Preserving Authentication Scheme for Drone-to-Drone Communications in Internet of Autonomous Things. *IEEE Trans. Mob. Comput.* **2026**, *25*, 1445–1456. <https://doi.org/10.1109/TMC.2025.3601369>.
35. Peivandizadeh, A.; Alasem, R.; Farhadi Moghadam, M.; Adarbah, H.Y.; Molavi, B.; Mohajerzadeh, A.; Noore, A. Lightweight and Efficient Protocol Based on ECDH for Securing Smart Grid Communication Infrastructure. *IEEE Access* **2025**, *13*, 123666–123681. <https://doi.org/10.1109/ACCESS.2025.3585982>.
36. Kurose, J.F.; Ross, K.W. *Computer Networking: A Top-Down Approach Edition*; Addison Wesley: Hoboken, NJ, USA, 2007; Volume 12.
37. Kim, C.; Kwon, D.; Park, Y.; Park, Y. Quantum-Resistant Three-Party Mutual Authentication Protocol for Industrial IoT Environments. *IEEE Internet Things J.* **2026**, *13*, 25380–25397.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.